Assignment 1 Report

Jayesh Narayanan

2023EE11048

Linear Regression:

Part 1:

Methodology:

To implement batch gradient descent for optimizing J(θ), I initialized the parameters θ_0 and θ_1 to zero and iteratively updated them using the full dataset. At each step, the mean squared error was computed, and the stopping criterion was defined as the absolute change in the cost function falling below a small threshold ε To determine an appropriate learning rate, I started with a relatively large value and progressively reduced it (by a factor of 10 each time) until the algorithm converged stably without overshooting. The stability criterion used was to check if it took more than 1000 epochs(iterations) to converge. This systematic approach allowed me to identify the largest suitable learning rate that guarantees convergence while maintaining efficiency.

Results:

Learning rate = 0.1, θ_0 = 6.2168, θ_1 = 29.0559

Stopping criterion: $J(\theta_{t+1}) - J(\theta_t) < 10^{-5}$

Part 2:

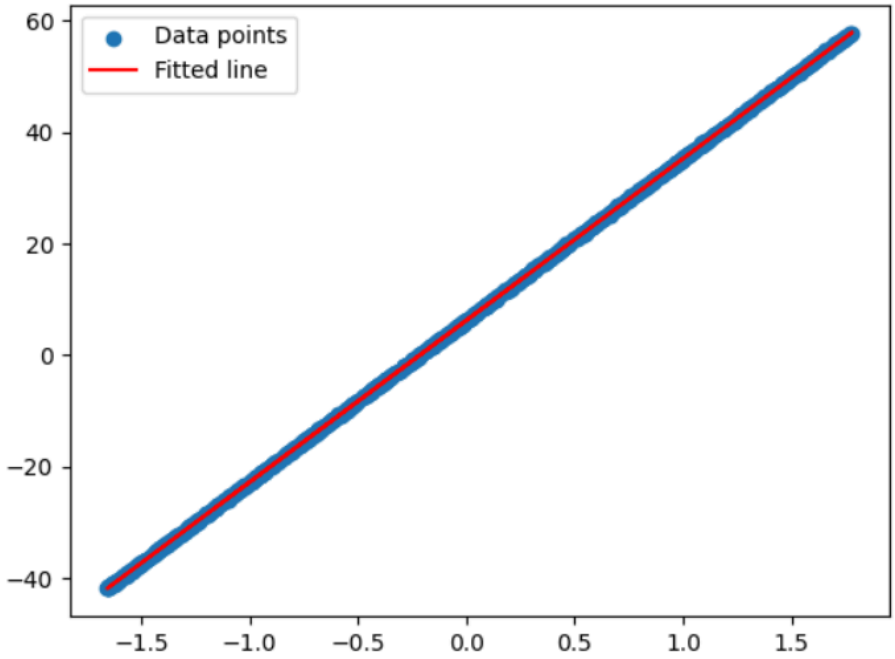


Figure 1: Plot of y= $\boldsymbol{\theta_1}$ x+$\boldsymbol{\theta_0}$ along with data points

Part 3:

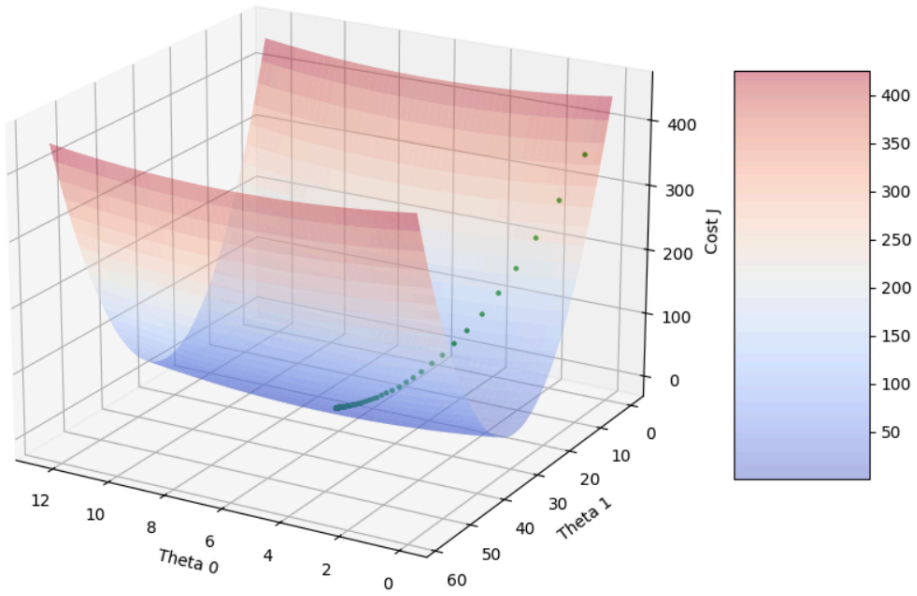


Figure 2: 3D mesh plot along with gradient descent path

Part 4:

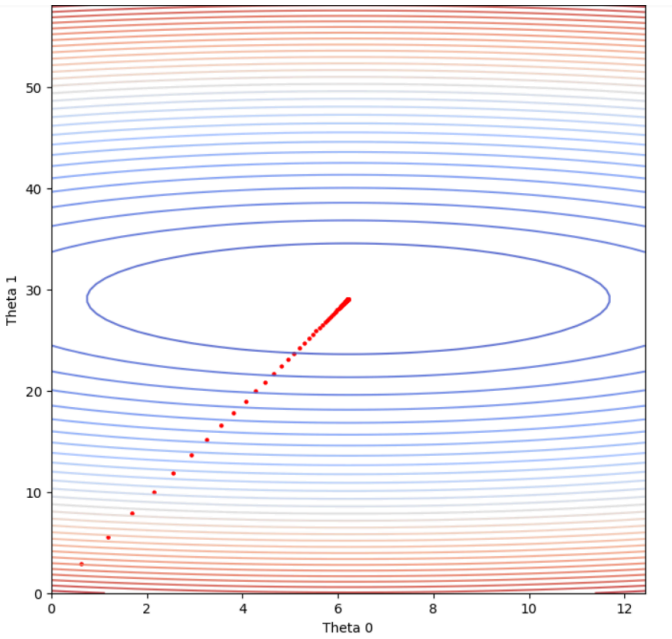
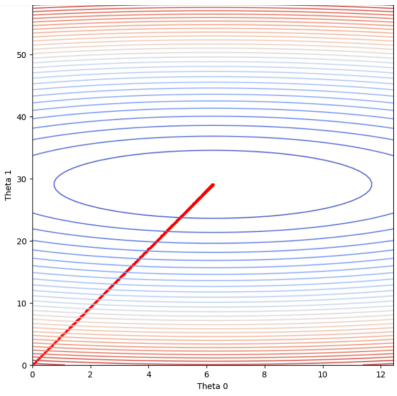
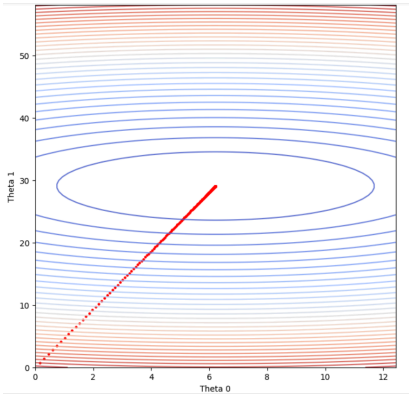


Figure 3: Current set of parameters

Part 5





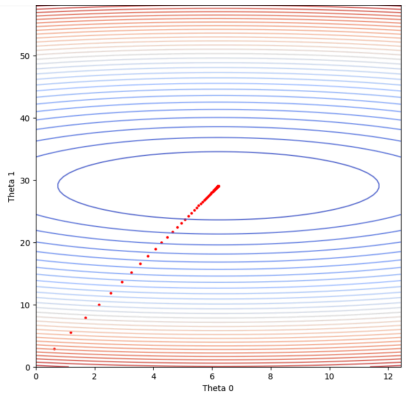


Figure 4: learning rate 0.01 *Figure 5: learning rate 0.025* *Figure 6: learning rate 0.01*

Conclusion

From the graphs we see that as the learning rate increases, the number of epochs required for convergence decreases. This happens because a higher learning rate causes larger parameter updates in each iteration, allowing the algorithm to reach values of θ that minimize the cost function more quickly. However, if the learning rate is too high, it may lead to overshooting or instability instead of faster convergence.

Sampling, Closed Form and Stochastic Gradient Descent

Part 1

Created synthetic data in the form of numpy arrays using .random.normal function

Part 2

Methodology:

I optimized a linear regression model with features (x1,x2) using stochastic/mini-batch gradient descent. Parameters were initialized to zero($\theta_0,\theta_1,\theta_2$)=(0,0,0)) and the learning rate was fixed at η=0.001. For each epoch, I randomly permuted the training set and iterated over contiguous mini-batches of size r∈{1,80,8000,800000}. I used a variable convergence criteria stated below.

Convergence criteria (batch-aware):

$$\frac{||\nabla J||^2}{M} < \varepsilon_r \quad \text{where, } \varepsilon_r \text{ is proportional to } 1/\sqrt{r}$$

Here $||\nabla J||^2$ is the Euclidean norm of the sum of batch gradients over an epoch (proxy for average gradient magnitude), r is the batch size, M is the dataset size, and c is a constant chosen to balance stability and speed. The reasoning for choosing this relationship is twofold: first, for small batch sizes the gradient estimates are noisier, so a looser stopping condition avoids excessive runtime spent chasing small fluctuations; second, the root proportionality ensures that the tolerance does not become unrealistically tight for large rrr, while still enforcing stricter convergence as the updates become smoother with larger batches. This balance allowed the algorithm to converge efficiently across different batch sizes, and a maximum epoch limit was also enforced as a safety check.

Values used:

- r=1, $\varepsilon_r = 10^{-1}$
- r=80, $\varepsilon_r = 10^{-1}/\sqrt{80}$
- r=8000, $\varepsilon_r = 10^{-1}/\sqrt{8000}$
- r=800000, $\varepsilon_r = 5\times10^{-2}$ (practical considerations)

Part 3:

The different algorithms converge to the same values i.e θ_0=3, θ_1=1, θ_2=2 However, the time and number of iterations taken to reach the convergent value is drastically different.

Relative speed:

$r_1 > r_{80} > r_{8000} > r_{800000}$

Number of iterations:

- r=1, epochs=1
- r=80, epochs=3
- r=8000, epochs=243
- r=800000, epochs=10082

Parameters obtained:

Closed form solution: θ_0=2.9994, θ_1=1.0010, θ_2=2.0008
Batch size 1 : θ_0=2.9767, θ_1=1.0188, θ_2=2.0112
Batch size 80 θ_0=2.9970, θ_1=0.9953, θ_2=2.0022
Batch size 8000 :θ_0=2.9955, θ_1=1.0017, θ_2=2.0006
Batch size 800000 :θ_0=2.8202, θ_1=1.0401, θ_2=1.9877

We observe that the closed form solution is the most accurate among the different algorithms

Part 4:

Test error v.s Train error:

Since the data were generated from the same linear model with independent, zero-mean Gaussian noise, and the estimator (gradient descent converged to the least-squares solution) is unbiased, the expected training and test mean-squared errors both approach the noise variance σ^2. In finite samples there is a small difference due to parameter estimation (degrees of freedom): the expected training MSE equals $(1-p/n)\sigma^2$ while the expected test MSE is approximately $(1+p/n)\sigma^2$, where p is the number of parameters and nnn the training size. For our experiment p=3and n=800,000, so p/n is negligible; therefore the observed training and test errors are effectively equal, as predicted for all the algorithms.

Part 5:

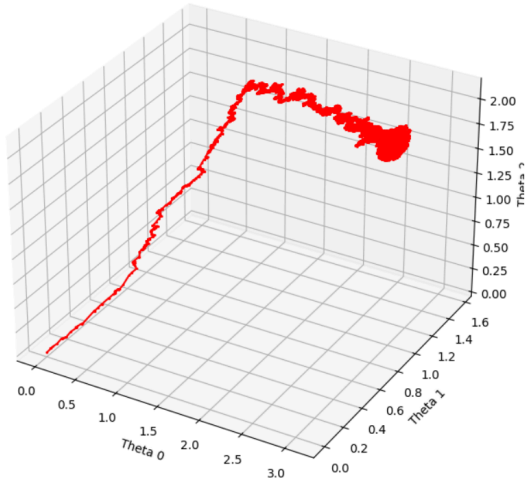


Figure 7: batch size=1

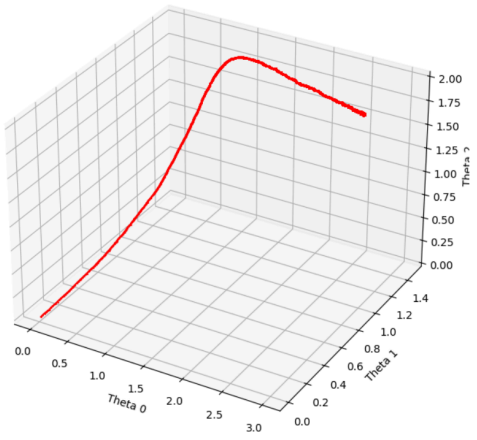


Figure 8: batch size=80

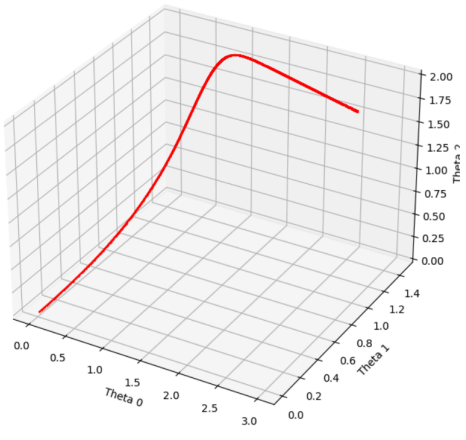


Figure 9: batch size=8000

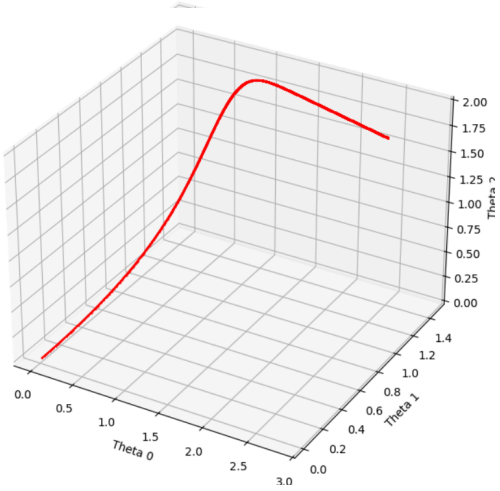


Figure 10: batch size= 800000

Batch r=1 (pure SGD, single-example updates):
The trajectory is *very noisy* — it looks like a jagged, high-frequency zig-zag or random-walk superimposed on an overall direction toward the optimum. The path frequently overshoots locally and corrects, producing many small loops. This makes the curve long in path-length and visibly rough in 3D. Intuition: every update uses one noisy sample, so the instantaneous gradient points in very different directions from step to step.

Batch r=80 (small mini-batch):
The trajectory becomes noticeably smoother than r=1 but still shows visible jitter. You'll see the overall descent is clearer (fewer tiny loops) while still having stochastic deviations around the main descent curve. Convergence per epoch is usually faster than r=1 (less wasted movement) but still noisy.

Batch r=800 (moderate mini-batch):
The path is much smoother and more directly follows the full-batch descent direction. Jitter is small; the movement looks like a smooth curve spiralling gradually into the minimum (if the surface is anisotropic) or a nearly straight line if the surface is well-conditioned.

Batch r=800000 (full batch / deterministic GD):
The trajectory is the **smoothest and most direct** — essentially the noiseless gradient-descent curve. If the cost surface is well-conditioned, the path will be a short smooth curve almost directly to the minimum. No random zig-zags appear.

Logistic Regression:

Part 1:

Methodology:

We implemented logistic regression using the Newton–Raphson optimization technique. The input dataset was first normalized feature-wise, and an intercept term was added to the design matrix. Parameters θ\thetaθ were initialized to zero and iteratively updated. In each iteration, we calculated the gradient of the log-likelihood and constructed the Hessian matrix, which was then inverted to compute the Newton step. The update rule $\theta \leftarrow \theta - H^{-1} \nabla J(\theta)$ was applied until the norm of parameter changes fell below a chosen threshold(10^{-6}), ensuring convergence. Finally, the learned parameters were used to plot the decision boundary, and the data points were visualized according to their true class labels.

Results:

Using the Newton–Raphson method, the logistic regression model converged in **8 iterations**.
The final learned parameters were:

$$\theta = (\theta_0 = 2.5577,\ \theta_1 = -2.7814,\ \theta_2 = 0.4672)$$

The decision boundary is given by:

$$\theta_0 x_1 + \theta_1 x_2 + \theta_2 = 0$$

where θ_2 is the intercept term.

The decision boundary was successfully plotted, separating the two classes in the feature space. The boundary visually aligns well with the data distribution, showing that the model has effectively distinguished between the two classes.

Part 2:

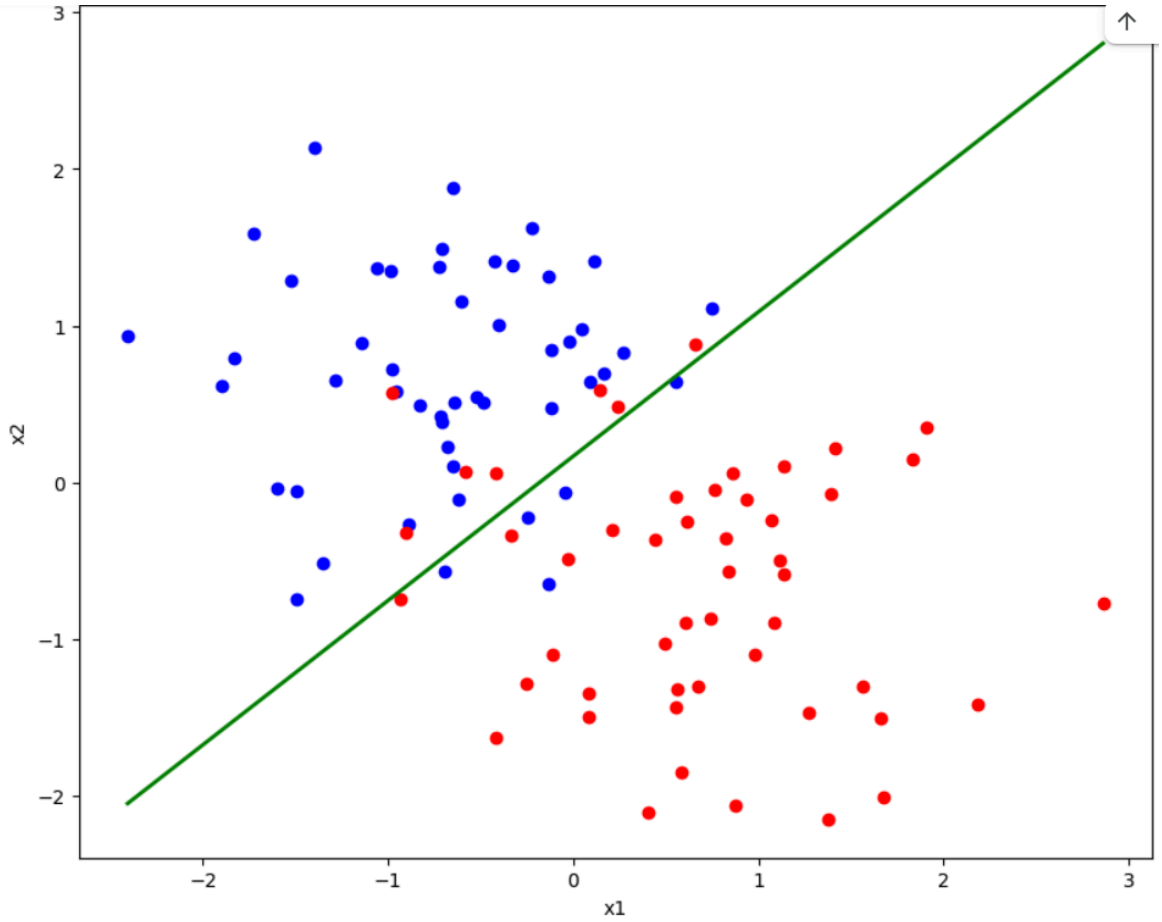


Figure 11: Plot of true labels along with decision boundary

Gaussian Discriminant Analysis

Linear decision boundary:

$\mu_0 = \frac{1}{N0} \sum_{y=0} x_i$

$\mu_1 = \frac{1}{N1} \sum_{y=1} x_i$

Covariance : $$\Sigma = \frac{1}{m} \sum_{i=1}^{m} \left(x^{(i)} - \mu_{y^{(i)}}\right)\left(x^{(i)} - \mu_{y^{(i)}}\right)^\top$$

Parametric vector: $$\theta = \Sigma^{-1}(\mu_1 - \mu_0)$$

Intercept term: $$\theta_0 = \tfrac{1}{2}(\mu_0^\top \Sigma^{-1}\mu_0 - \mu_1^\top \Sigma^{-1}\mu_1) + \log\frac{\phi}{1-\phi}$$

Decision boundary equation: $$\theta^\top x + \theta_0 = 0$$

Using the above equations,

μ_0=[-0.7552 0.6850] , μ_1=[0.7552 -0.6850]

Σ= [[1.0101 -0.5453]
[-0.5453 1.0101]]

the values of parameters obtained are

θ_0= $3.3306*10^{-16}$,θ_1= 1.0770, θ_2= -0.7749

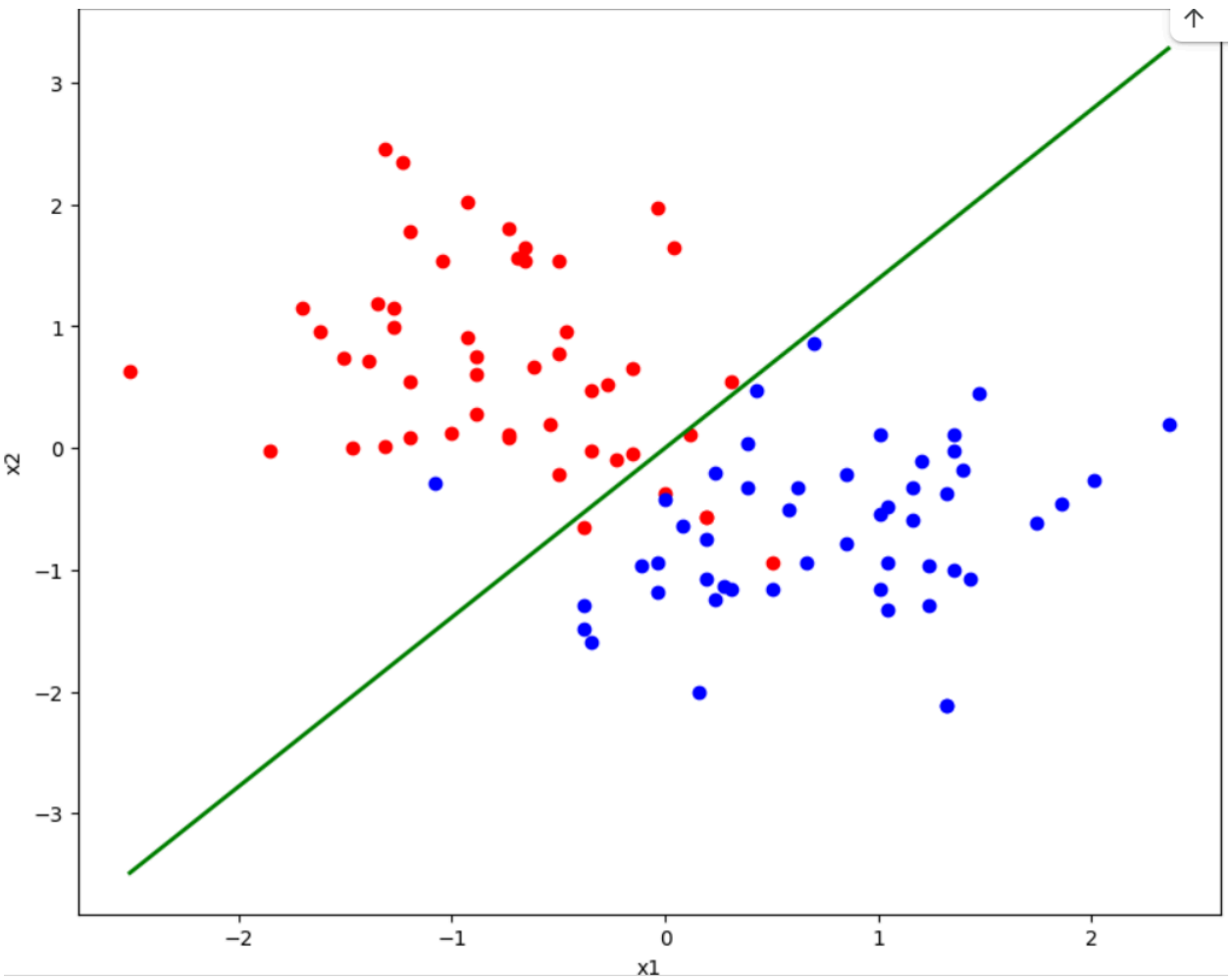


Figure 12: Plot x_2=1.389x_1+4.291e-16 along with data points Alaska:Red, Canada:Blue

Quadratic decision boundary:

$\mu_0 = \frac{1}{N0} \sum_{y=0} x_i$

$\mu_1 = \frac{1}{N1} \sum_{y=1} x_i$

Covariance_0: $$\Sigma_0 = \frac{1}{m_0} \sum_{i=1}^{m} 1\{y^{(i)} = 0\} (x^{(i)} - \mu_0)(x^{(i)} - \mu_0)^\top$$

Covariance_1: $$\Sigma_1 = \frac{1}{m_1} \sum_{i=1}^{m} 1\{y^{(i)} = 1\} (x^{(i)} - \mu_1)(x^{(i)} - \mu_1)^\top$$

Decision function: $$g(x) = x^\top A\, x + b^\top x + c$$

Parameters:

$$A = \tfrac{1}{2}\left(\Sigma_0^{-1} - \Sigma_1^{-1}\right), \qquad b = \Sigma_1^{-1}\mu_1 - \Sigma_0^{-1}\mu_0,$$

$$c = -\tfrac{1}{2}\left(\mu_1^\top \Sigma_1^{-1}\mu_1 - \mu_0^\top \Sigma_0^{-1}\mu_0\right) - \tfrac{1}{2}\log\frac{\det \Sigma_1}{\det \Sigma_0} + \log\frac{\phi}{1-\phi}.$$

Using the above equations,

μ_0=[-0.7552 0.6850] , μ_1=[0.7552 -0.6850]

Σ_0= [[0.3893 -0.1580] Σ_1= [[0.4872 0.1121]
[-0.1580 0.6609]] [0.1121 0.4219]]

the values of parameters obtained are

A=[[0.3289 0.6305] B=[3.7316 -2.8024] C=-0.5712
 [0.6305 -0.4243]]

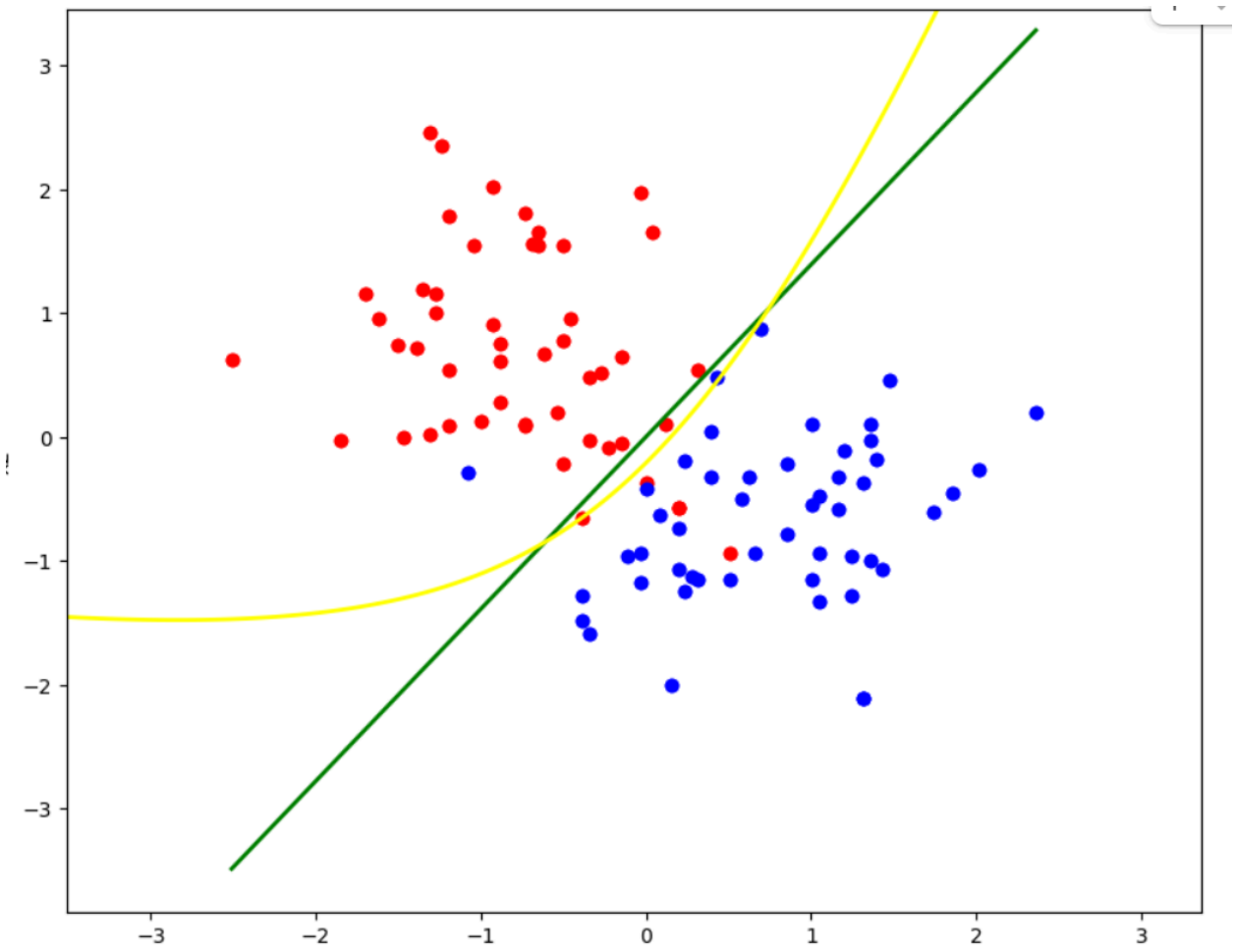


Figure 13: Quadratic and linear decision boundary with data points

Analysis:

The linear boundary — computed under the shared-covariance assumption — is a straight line and reflects differences in the class means only. The quadratic boundary — computed allowing distinct covariances for each class — is generally curved and captures differences in class spread and orientation. If Σ_0 and Σ_1 are similar, both boundaries look similar; if they differ, the quadratic boundary bends to follow class-specific covariance structure. Quantitatively, one can compare accuracy or log-likelihood on a held-out set; qualitatively, overlaying both boundaries on the scatter plot reveals where QDA corrects classification errors that the linear separator makes.